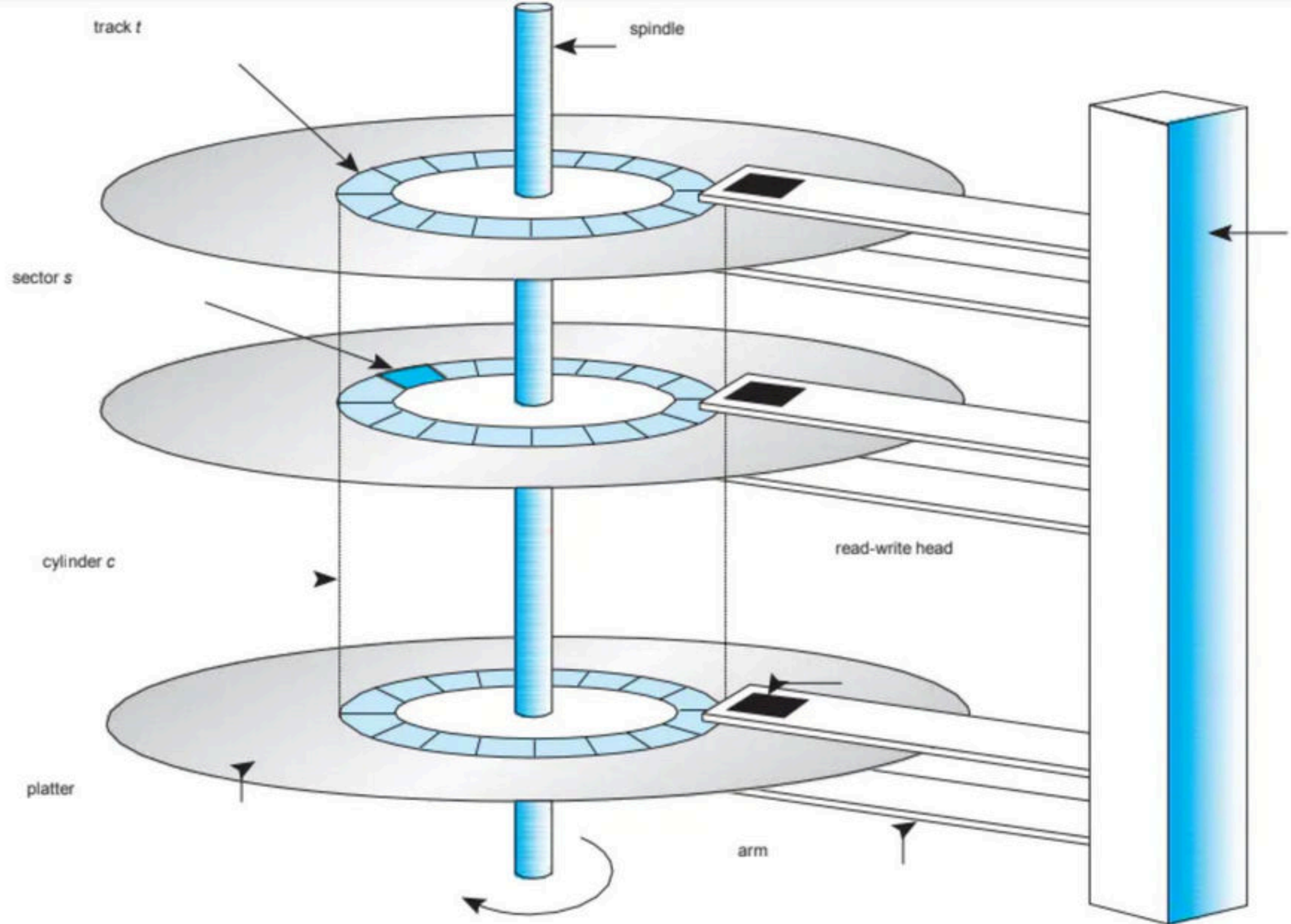




Understanding Fork command

Foundation Course on Operating Systems for GATE 2022





Disk scheduling

- One of the responsibilities of the operating system is to use the hardware efficiently. For the disk drives, efficiency means having fast access time and large disk bandwidth.
- The disk **bandwidth** is the total number of bytes transferred, divided by the total time between the first request for service and the completion of the last transfer. We can improve both the access time and the bandwidth by managing the order in which disk I/O requests are serviced.

Whenever a process needs I/O to or from the disk, it issues a system call to the operating system. The request specifies several pieces of information:

- Whether this operation is input or output
 - What is the disk address for the transfer
 - What is the memory address for the transfer
 - What is the number of sectors to be transferred
-
- If the desired disk drive and controller are available, the request can be serviced immediately.
 - If the drive or controller is busy, any new requests for service will be placed in the queue of pending requests for that drive.
 - Thus, when one request is completed, the operating system chooses which pending request to service next. How does the operating system make this choice? Any one of several disk-scheduling algorithms can be used.

Break

FCFS (First Come First Serve)

The simplest form of disk scheduling is, of course, the first-come, first-served (FCFS) algorithm. In FCFS, the requests are addressed in the order they arrive in the disk queue. This algorithm is intrinsically fair, but it generally does not provide the fastest service.

Advantages:

- Easy to understand easy to use
- Every request gets a fair chance
- no starvation (may suffer from convoy effect)

Disadvantages:

- Does not try to optimize seek time (extra seek movements)
- May not provide the best possible service

Q If the disk head is located initially at 32, find the number of disk moves required with FCFS if the disk queue of I/O blocks requests are 98, 37, 14, 124, 65, 67. **(NET-DEC-2012)**

(A) 239

(B) 310

(C) 321

(D) 325

Q Assuming that the disk head is located initially at 32, find the number of disk moves required with FCFS if the disk queue of I/O block requests are 98, 37, 14, 124, 65, 67: **(NET-JUNE-2013)**

(A) 310

(B) 324

(C) 320

(D) 321

Q If the Disk head is located initially at track 32, find the number of disk moves required with FCFS scheduling criteria if the disk queue of I/O blocks requests are: 98, 37, 14, 124, 65, 67 (**NET-JULY-2016**)

- a) 320 b) 322 c) 321 d) 319

Break

SSTF Scheduling

- It seems reasonable to service all the requests close to the current head position before moving the head far to service other REQUESTS. This assumption is the basis for the SSTF algorithm.
- In SSTF (Shortest Seek Time First), the request nearest to the disk arm will get executed first i.e. requests having shortest seek time are executed first.
- Although the SSTF algorithm is a substantial improvement over the FCFS algorithm, it is not optimal. In the example, we can do better by moving the head from 53 to 37, even though the latter is not closest, and then to 14, before turning around to service 65, 67, 98, 122, 124, and 183. This strategy reduces the total head movement to 208 cylinders.

Advantages:

- Seek movements decreases
- Average Response Time decreases
- Throughput increases

Disadvantages:

- Overhead to calculate the closest request.
- Can cause Starvation for a request which is far from the current location of the header
- High variance of response time as SSTF favours only some requests
- SSTF scheduling is essentially a form of shortest-job-first (SJF) scheduling; and like SJF scheduling, it may cause starvation of some requests.

Q Consider a disk system with 100 cylinders. The requests to access the cylinders occur in following sequence 4, 34, 10, 7, 19, 73, 2, 15, 6, 20. Assuming that the head is currently at cylinder 50, what is the time taken to satisfy all requests if it takes 1ms to move from one cylinder to adjacent one and shortest seek time first policy is used? **(GATE-2009) (1 Marks)**

(A) 95 ms **(B)** 119 ms **(C)** 233 ms **(D)** 276 ms

Q Suppose a disk has 201 cylinders, numbered from 0 to 200. At some time, the disk arm is at cylinder 100, and there is a queue of disk access requests for cylinders 30, 85, 90, 100, 105, 110, 135 and 145. If Shortest-Seek Time First (SSTF) is being used for scheduling the disk access, the request for cylinder 90 is serviced after servicing _____ number of requests. **(GATE-2014) (1 Marks)**

Q Consider an imaginary disk with 40 cylinders. A request come to read a block on cylinder 11. While the seek to cylinder 11 is in progress, new requests come in for cylinders 1, 36, 16, 34, 9 and 12 in that order. The number of arm motions using shortest seek first algorithm is **(NET-DEC-2014)**

(A) 111

(B) 112

(C) 60

(D) 61

Q Consider a disk queue with requests for I/O to blocks on cylinders 98, 183, 37, 122, 14, 124, 65, 67. Suppose SSTF disk scheduling algorithm implemented to meet the requests then the total number of head movements are _____ if the disk head is initially at 53. **(NET-NOV-2017)**

a) 224

b) 248

c) 236

d) 240

Q A disk drive has 100 cylinders, numbered 0 to 99. Disk requests come to the disk driver for cylinders 12, 26, 24, 4, 42, 8 and 50 in that order. The driver is currently serving a request at cylinder 24. A seek takes 6 msec per cylinder moved. How much seek time is needed for shortest seek time first (SSTF) algorithm? **(NET-JUNE-2015)**

a) 0.984 sec

b) 0.396 sec

c) 0.738 sec

d) 0.42 sec

Break

SCAN

The disk arm starts at one end of the disk and moves towards the other end, servicing requests as it reaches each track, until it gets to the other end of the disk. At the other end, the direction of head movement is reversed, and servicing continues. The head continuously scans back and forth across the disk. The SCAN algorithm is sometimes called the Elevator algorithm, since the disk arm behaves just like an elevator in a building, first servicing all the requests going up and then reversing to service requests the other way.

- If a request arrives in the queue just in front of the head, it will be serviced almost immediately; a request arriving just behind the head will have to wait until the arm moves to the end of the disk, reverses direction, and comes back.
- Assuming a uniform distribution of requests for cylinders, consider the density of requests when the head reaches one end and reverses direction. At this point, relatively few requests are immediately in front of the head since these cylinders have recently been serviced. The heaviest density of requests is at the other end of the disk. These requests have also waited the longest.

Advantages:

- Simple easy to understand and use
- No starvation but more wait for some random process
- Low variance and Average response time

Disadvantages:

- Long waiting time for requests for locations just visited by disk arm.
- Unnecessary move to the end of the disk, even if there is no request.

Q On a disk with 1000 cylinders (0 to 999) find the number of tracks, the disk arm must move to satisfy all the requests in the disk queue. Assume the last request service was at track 345 and the head is moving toward track 0. The queue in FIFO order contains requests for the following tracks: 123, 874, 692, 475, 105, 376 (Assume SCAN algorithm) **(NET-DEC-2012)**

(A) 2013 **(B)** 1219 **(C)** 1967 **(D)** 1507

Q Consider a disk queue with I/O requests on the following cylinders in their arriving order: 6, 10, 12, 54, 97, 73, 128, 15, 44, 110, 34, 45. The disk head is assumed to be at cylinder 23 and moving in the direction of decreasing number of cylinders. Total number of cylinders in the disk is 150. The disk head movement using SCAN-scheduling algorithm is: **(NET-JAN-2017)**

a) 172 b) 173 c) 227 d) 228

Q Suppose the following disk request sequence (track numbers) for a disk with 100 tracks is given: 45, 20, 90, 10, 50, 60, 80, 25, 70. Assume that the initial position of the R/W head is on track 50. The additional distance that will be traversed by the R/W head when the Shortest Seek Time First (SSTF) algorithm is used compared to the SCAN (Elevator) algorithm (assuming that SCAN algorithm moves towards 100 when it starts execution) is _____ tracks **(GATE-2015) (2 Marks)**

Q Consider a disk queue with request for input/output to block on cylinders 98, 183, 37, 122, 14, 124, 65, 67 in that order. Assume that disk head is initially positioned at cylinder 53 and moving towards cylinder number 0. The total number of head movements using Shortest Seek Time First (SSTF) and SCAN algorithms are respectively (**NET-DEC-2013**)

(A) 236 and 252 cylinders

(B) 640 and 236 cylinders

(C) 235 and 640 cylinders

(D) 235 and 252 cylinders

Break

C-SCAN Scheduling

Circular-scan is a variant of SCAN designed to provide a more uniform wait time. Like SCAN, C-SCAN moves the head from one end of the disk to the other, servicing requests along the way. When the head reaches the other end, however, it immediately returns to the beginning of the disk without servicing any requests on the return trip.

Advantages:

- Provides more uniform wait time compared to SCAN
- Better response time compared to scan

Disadvantage:

- More seeks movements in order to reach starting position

Q Consider the situation in which the disk read/write head is currently located at track 45 (of tracks 0-255) and moving in the positive direction. Assume that the following track requests have been made in this order: 40, 67, 11, 240, 87. What is the order in which C-SCAN would service these requests and what is the total seek distance?

- (A) 600 (B) 810 (C) 505 (D) 550

Break

LOOK Scheduling

It is similar to the SCAN disk scheduling algorithm except the difference that the disk arm in spite of going to the end of the disk goes only to the last request to be serviced in front of the head and then reverses its direction from there only. Thus, it prevents the extra delay which occurred due to unnecessary traversal to the end of the disk.

Advantage: -

- Better performance compared to SCAN
- Should be used in case to less load

Disadvantage: -

- Overhead to find the last request
- Should not be used in case of more load.

Break

C LOOK

As LOOK is similar to SCAN algorithm, in similar way, C-LOOK is similar to C-SCAN disk scheduling algorithm. In C-LOOK, the disk arm in spite of going to the end goes only to the last request to be serviced in front of the head and then from there goes to the other end's last request. Thus, it also prevents the extra delay which occurred due to unnecessary traversal to the end of the disk.

Advantage: -

- Provides more uniform wait time compared to LOOK
- Better response time compared to LOOK

Disadvantage: -

- Overhead to find the last request and go to initial position is more
- Should not be used in case of more load.

Q Consider the situation in which the disk read/write head is currently located at track 45 (of tracks 0-255) and moving in the positive direction. Assume that the following track requests have been made in this order: 40, 67, 11, 240, 87. What is the order in which optimized C-SCAN would service these requests and what is the total seek distance?**(GATE-2015) (2 Marks)**

(A) 600

(B) 810

(C) 505

(D) 550

Q Which of the following statements is not true about disk-arm scheduling algorithms? **(NET-JUNE-2014)**

- (A)** SSTF (shortest seek time first) algorithm increases performance of FCFS.
- (B)** The number of requests for disk service are not influenced by file allocation method.
- (C)** Caching the directories and index blocks in main memory can also help in reducing disk arm movements.
- (D)** SCAN and C-SCAN algorithms are less likely to have a starvation problem.

Q In _____ disk scheduling algorithm, the disk head moves from one end to other end of the disk, serving the requests along the way. When the head reaches the other end, it immediately returns to the beginning of the disk without serving any requests on the return trip.

(NET-NOV-2017)

- a) LOOK b) SCAN c) C-LOOK d) C-SCAN

Q Suppose the following disk request sequence (track numbers) for a disk with 100 tracks is given: 45, 20, 90, 10, 50, 60, 80, 25, 70. Assume that the initial position of the R/W head is on track 50. The additional distance that will be traversed by the R/W head when the Shortest Seek Time First (SSTF) algorithm is used compared to the SCAN (Elevator) algorithm (assuming that SCAN algorithm moves towards 100 when it starts execution) is _____ tracks (**GATE-2015**) (**2 Marks**)

Q Consider a disk queue with requests for I/O to blocks on cylinders 47, 38, 121, 191, 87, 11, 92, 10. The C-LOOK scheduling algorithm is used. The head is initially at cylinder number 63, moving towards larger cylinder numbers on its servicing pass. The cylinders are numbered from 0 to 199. The total head movement (in number of cylinders) incurred while servicing these requests is _____. **(GATE-2018)**
(2 Marks)

Break

Conclusion

Selection of a Disk-Scheduling Algorithm

- Given so many disk-scheduling algorithms, how do we choose the best one? SSTF is common and has a natural appeal because it increases performance over FCFS. SCAN and C-SCAN perform better for systems that place a heavy load on the disk, because they are less likely to cause a starvation problem.
- With any scheduling algorithm, however, performance depends heavily on the number and types of requests. For instance, suppose that the queue usually has just one outstanding request. Then, all scheduling algorithms behave the same, because they have only one choice of where to move the disk head: they all behave like FCFS scheduling.

Q Consider an operating system capable of loading and executing a single sequential user process at a time. The disk head scheduling algorithm used is First Come First Served (FCFS). If FCFS is replaced by Shortest Seek Time First (SSTF), claimed by the vendor to give 50% better benchmark results, what is the expected improvement in the I/O performance of user programs? **(GATE-2004) (1 Marks)**

(A) 50%

(B) 40%

(C) 25%

(D) 0%

Q Which of the following disk scheduling strategies is likely to give the best throughput? **(GATE-1999) (1 Marks)**

(a) Farthest cylinder next

(c) First come first served

(b) Nearest cylinder next

(d) Elevator algorithm

Q Match the following (NET-DEC-2004)

(a) Disk scheduling	(1) Round robin
(b) Batch processing	(2) Scan
(c) Time sharing	(3) LIFO
(d) Interrupt processing	(4) FIFO

Codes:

(A)	a-3	b-4	c-2	d-1
(B)	a-4	b-3	c-2	d-1
(C)	a-2	b-4	c-1	d-3
(D)	a-3	b-4	c-1	d-2

Use Referral Code KGYT for Unacademy Plus to Get minimum 10% Discount

Q The maximum amount of information that is available in one portion of the disk access arm for a removal disk pack (without further movement of the arm with multiple heads) **(NET-DEC-2011)**

(A) a plate of data

(B) a cylinder of data

(C) a track of data

(D) a block of data

Q Consider the input/output (I/O) requests made at different instants of time directed at a hypothetical disk having 200 tracks as given in the following table

Serial No	1	2	3	4	5
Track No	12	85	40	100	75
Time of arrival	65	80	110	100	175

Assume that :

Current head position is at track no. 65 Direction of last movement is towards higher numbered tracks Current clock time is 160 milliseconds Head movement time per track is 1 millisecond. “look” is a variant of “SCAN” disk- arm scheduling algorithm. In this algorithm, if no more I/O requests are left in current direction, the disk head reverses its direction. The seek times in Shortest Seek First (SSF) and “look” disk-arm scheduling algorithms respectively are:

(NET-SEP-2013)

(A) 144 and 123 milliseconds

(C) 149 and 124 milliseconds

(B) 143 and 123 milliseconds

(D) 256 and 186 milliseconds

Use Referral Code KGYT for Unacademy Plus to Get minimum 10% Discount

Q Consider a storage disk with 4 platters (numbered as 0, 1, 2 and 3), 200 cylinders (numbered as 0, 1, ..., 199), and 256 sectors per track (numbered as 0, 1, ... 255). The following 6 disk requests of the form [sector number, cylinder number, platter number] are received by the disk controller at the same time:

[120, 72, 2], [180, 134, 1], [60, 20, 0], [212, 86, 3], [56, 116, 2], [118, 16, 1]

Currently head is positioned at sector number 100 of cylinder 80, and is moving towards higher cylinder numbers. The average power dissipation in moving the head over 100 cylinders is 20 milliwatts and for reversing the direction of the head movement once is 15 milliwatts. Power dissipation associated with rotational latency and switching of head between different platters is negligible.

The total power consumption in milliwatts to satisfy all of the above disk requests using the Shortest Seek Time First disk scheduling algorithm is _____. **(GATE-2018) (2 Marks)**

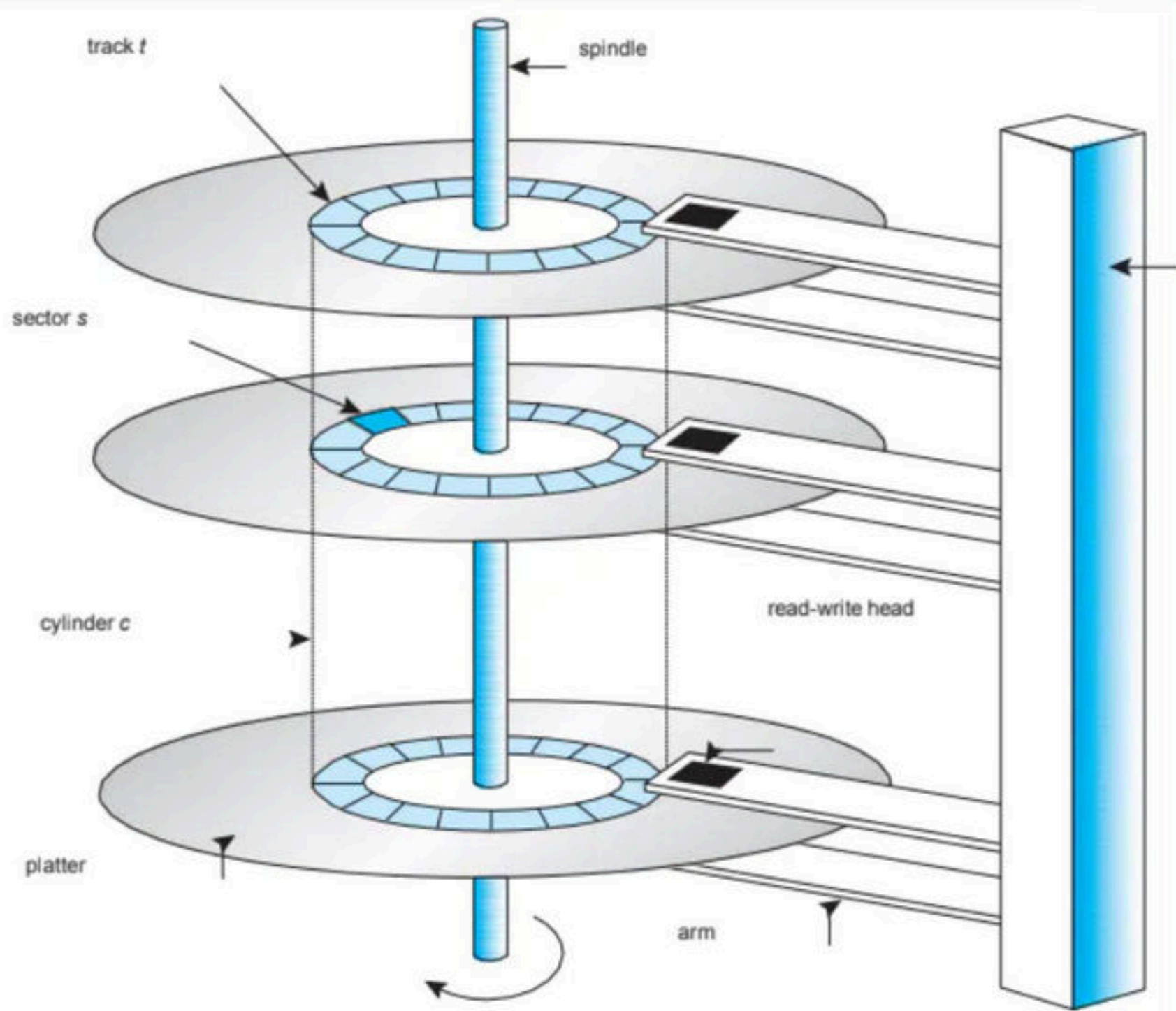
Q For a magnetic disk with concentric circular tracks, the seek latency is not linearly proportional to the seek distance due to **(GATE-2008) (1 Marks)**

- a)** non-uniform distribution of requests
- b)** arm starting and stopping inertia
- c)** higher capacity of tracks on the periphery of the platter
- d)** use of unfair arm scheduling policies

Break

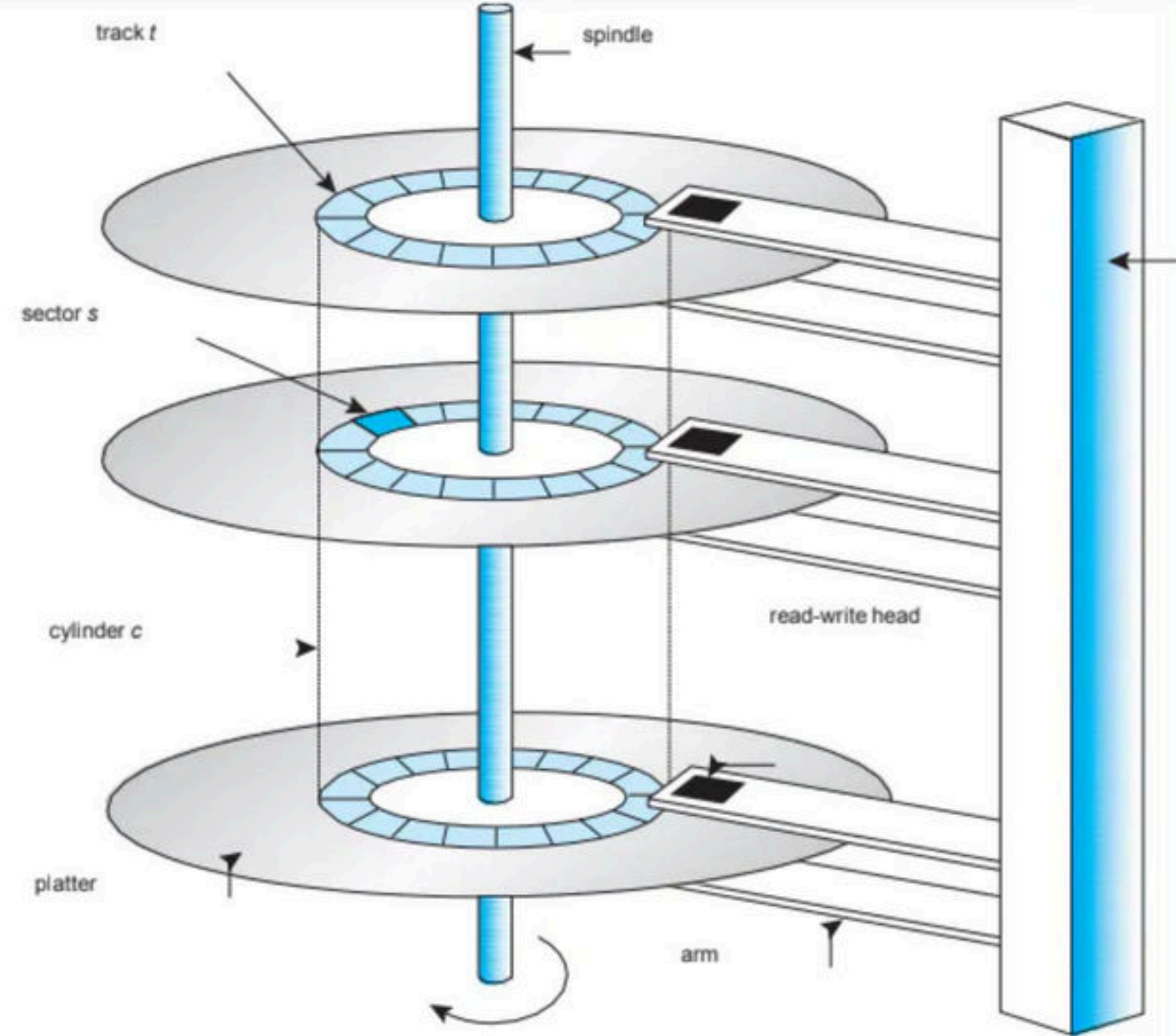
Transfer time

- Magnetic disks provide the bulk of secondary storage for modern computer systems. Each disk platter has a flat circular shape, like a CD. Common platter diameters range from 1.8 to 3.5 inches. The two surfaces of a platter are covered with a magnetic material. We store information by recording it magnetically on the platters.
- A read – write head “flies” just above each surface of every platter. The heads are attached to a disk arm that moves all the heads as a unit. The surface of a platter is logically divided into circular tracks, which are subdivided into sectors. The set of tracks that are at one arm position makes up a cylinder. There may be thousands of concentric cylinders in a disk drive, and each track may contain hundreds of sectors.





- When the disk is in use, a drive motor spins it at high speed. Most drives rotate 60 to 250 times per second, specified in terms of rotations per minute (**RPM**). Common drives spin at 5,400, 7,200, 10,000, and 15,000 RPM.
- Total transfer time has two parts. The **positioning time**, or **random-access time**, consists of two parts: the time necessary to move the disk arm to the desired cylinder, called the **seek time**, and the time necessary for the desired sector to rotate to the disk head, called the **rotational latency**.



- Total Transfer Time = Seek Time + Rotational Latency + Transfer Time
- **Seek Time:** - It is a time taken by Read/Write header to reach the correct track. (Always given in question)
- **Rotational Latency:** - It is the time taken by read/Write header during the wait for the correct sector. In general, it's a random value, so far average analysis, we consider the time taken by disk to complete half rotation.
- **Transfer Time:** - it is the time taken by read/write header either to read or write on a disk. In general, we assume that in 1 complete rotation, header can read/write the either track, so
- total time will be = (File Size/Track Size) *time taken to complete one revolution.

Q For a magnetic disk with concentric circular tracks, the seek latency is not linearly proportional to the seek distance due to **(GATE-2008) (2 Marks)**

- (A)** non-uniform distribution of requests
- (B)** arm starting and stopping inertia
- (C)** higher capacity of tracks on the periphery of the platter
- (D)** use of unfair arm scheduling policies

Q Consider a disk where there are 512 tracks, each track is capable of holding 128 sector and each sector holds 256 bytes, find the capacity of the track and disk and number of bits required to reach correct track, sector and disk.

Use Referral Code **KGYT for Unacademy Plus to Get minimum 10% Discount**

Q Consider a disk pack with 16 surfaces, 128 tracks per surface and 256 sectors per track. 512 bytes of data are stored in a bit serial manner in a sector. The capacity of the disk pack and the number of bits required to specify a particular sector in the disk are respectively **(GATE-2006) (2 Marks)**

(A) 256 Mbyte, 19 bits

(B) 256 Mbyte, 28 bits

(C) 512 Mbyte, 20 bits

(D) 64 Gbyte, 28 bit

Break

Q consider a disk where each sector contains 512 bytes and there are 400 sectors per track and 1000 tracks on the disk. If disk is rotating at speed of 1500 RPM, find the total time required to transfer file of size 1 MB. Suppose seek time is 4ms?

Use Referral Code KGYT for Unacademy Plus to Get minimum 10% Discount

Q Consider a disk with 16384 bytes per track having a rotation time of 16 msec and average seek time of 40 msec. What is the time in msec to read a block of 1024 bytes from this disk? **(NET-DEC-2015)**

a) 57 sec

b) 49 sec

c) 48 sec

d) 17 sec

Q Consider a system with 8 sector per track and 512 bytes per sector. Assume that disk rotates at 3000 rpm and average seek time is 15ms standard. Find total time required to transfer a file which requires 8 sectors to be stored.

a) Assume contiguous allocation

b) Assume Non- contiguous allocation

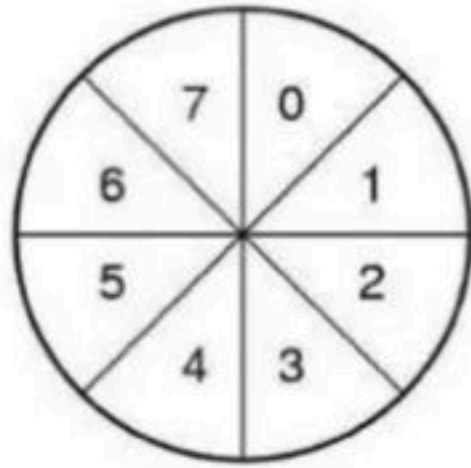
Q Consider a disk pack with a seek time of 4 milliseconds and rotational speed of 10000 rotations per minute (RPM). It has 600 sectors per track and each sector can store 512 bytes of data. Consider a file stored in the disk. The file contains 2000 sectors. Assume that every sector access necessitates a seek, and the average rotational latency for accessing each sector is half of the time for one complete rotation. The total time (in milliseconds) needed to read the entire file is _____. **(GATE-2015) (2 Marks)**

Q Consider a typical disk that rotates at 15000 rotations per minute (RPM) and has a transfer rate of 50×10^6 bytes/sec. If the average seek time of the disk is twice the average rotational delay and the controller's transfer time is 10 times the disk transfer time, the average time (in milliseconds) to read or write a 512 byte sector of the disk is _____ **(GATE-2015) (2 Marks)**

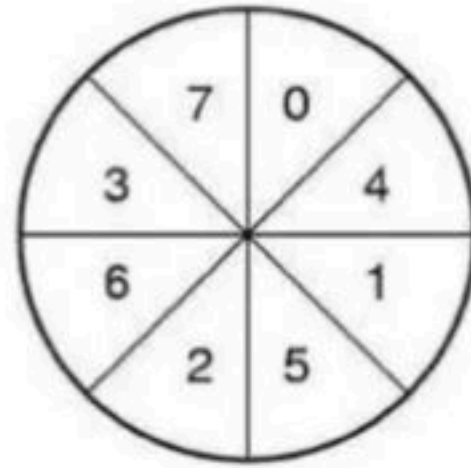
Q An application loads 100 libraries at start-up. Loading each library requires exactly one disk access. The seek time of the disk to a random location is given as 10 milli second. Rotational speed of disk is 6000 rpm. If all 100 libraries are loaded from random locations on the disk, how long does it take to load all libraries? (The time to transfer data from the disk block once the head has been positioned at the start of the block may be neglected) **(GATE-2011) (2 Marks)**

(A) 0.50 s **(B)** 1.50 s **(C)** 1.25 s **(D)** 1.00 s

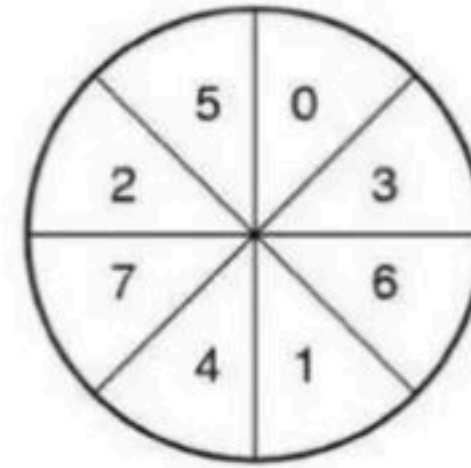
- **Single interleaving:** - in 2 rotation we read 1 track
- **Double interleaving:** - in 2.75 rotation we read 1 track



(a)



(b)



(c)

- No interleaving
- Single interleaving
- Double interleaving

File allocation methods

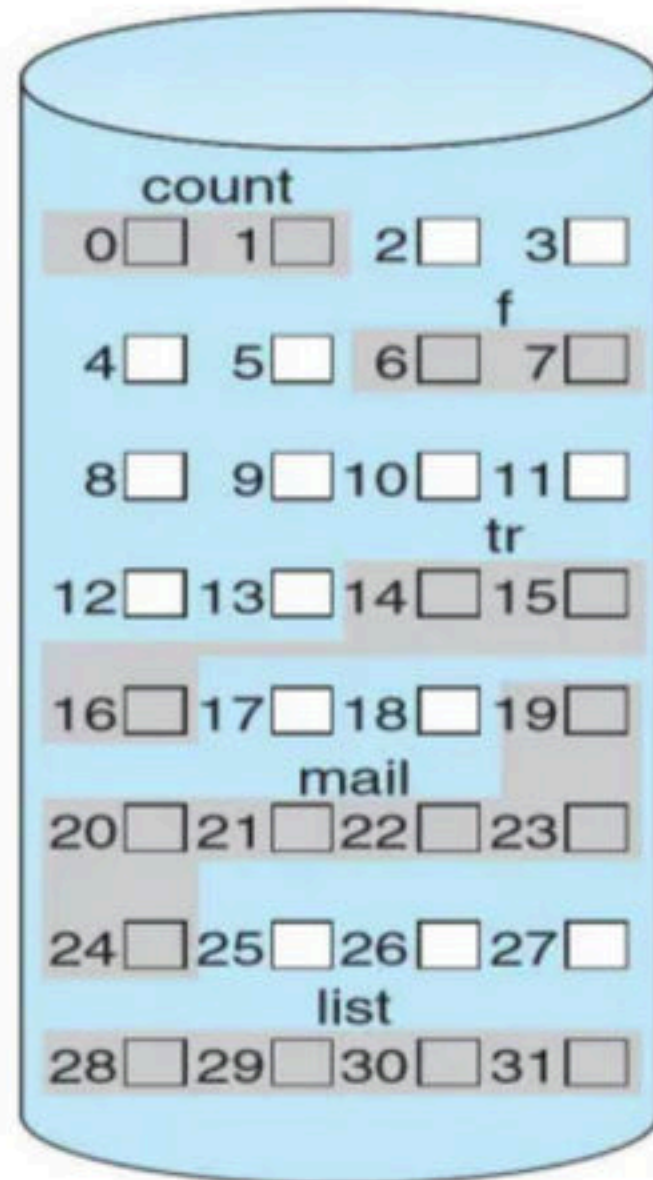
The main aim of file allocation problem is how disk space is utilized effectively and files can be accessed quickly. Three major methods of allocating disk space are in wide use:

- **Contiguous**
- **Linked**
- **Indexed**

Each method has advantages and disadvantages. Although some systems support all three, it is more common for a system to use one method for all files.

Contiguous Allocation

- Contiguous allocation requires that each file occupy a set of contiguous blocks on the disk. Disk addresses define a linear ordering on the disk. With this ordering, assuming that only one job is accessing the disk, accessing block $b + 1$ after block b normally requires no head movement.



directory

file	start	length
count	0	2
tr	14	3
mail	19	6
list	28	4
f	6	2

- **Advantage**

- Accessing a file that has been allocated contiguously is easy. Thus, both sequential and direct access can be supported by contiguous allocation.

- **Disadvantage**

- Suffer from the problem of external fragmentation.
- Suffer from huge amount of external fragmentation.
- Another problem with contiguous allocation file modification

Q Suppose there are six files $F_1, F_2, F_3, F_4, F_5, F_6$ with corresponding sizes 150 KB, 225 KB, 75 KB, 60 KB, 275 KB and 65 KB respectively. The files are to be stored on a sequential device in such a way that optimizes access time. In what order should the files be stored? **(NET-NOV-2017)**

A) $F_5, F_2, F_1, F_3, F_6, F_4$

b) $F_4, F_6, F_3, F_1, F_2, F_5$

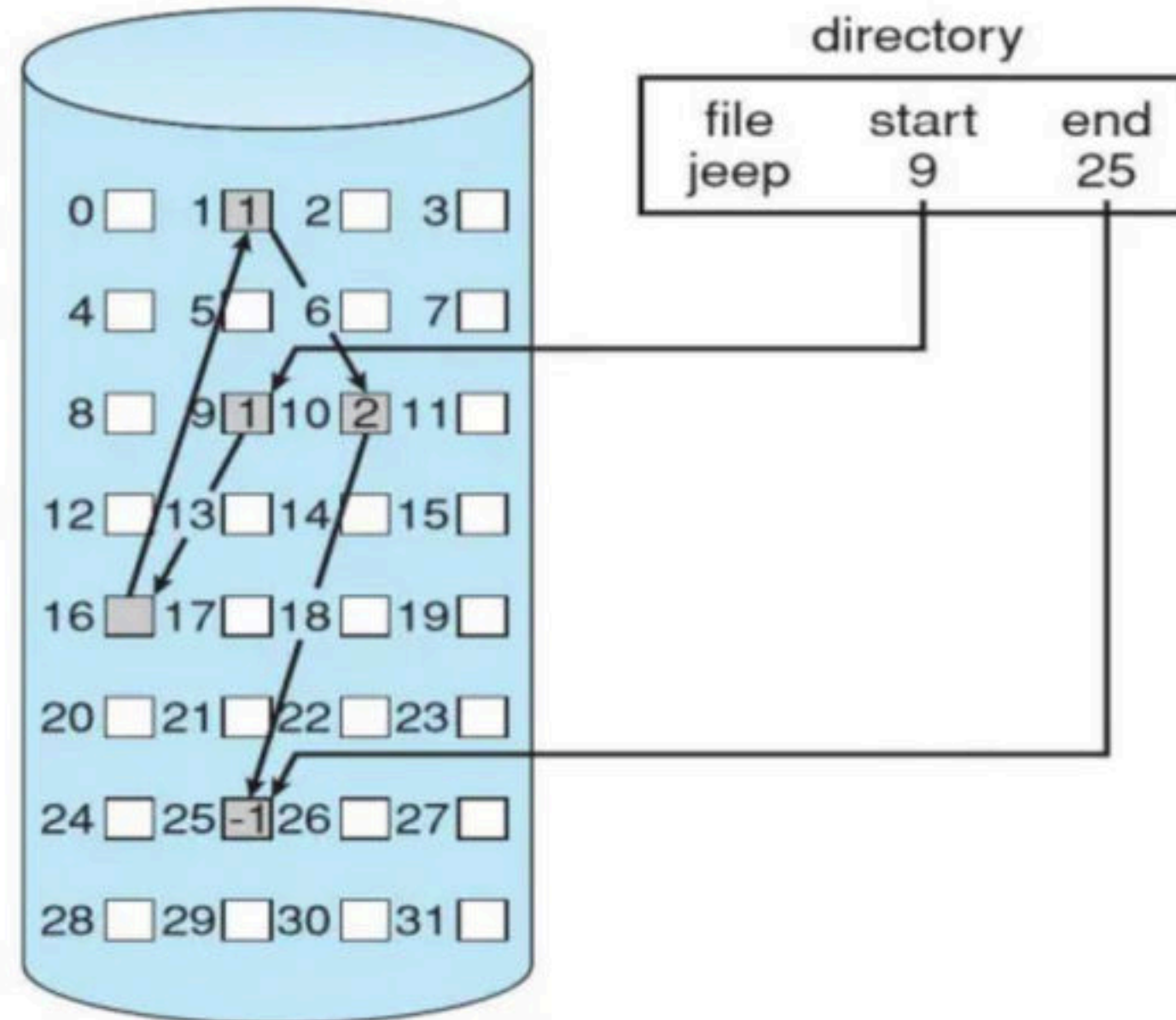
c) $F_1, F_2, F_3, F_4, F_5, F_6$

d) $F_6, F_5, F_4, F_3, F_2, F_1$

Break

Linked Allocation

- Linked allocation solves all problems of contiguous allocation. With linked allocation, each file is a linked list of disk blocks; the disk blocks may be scattered anywhere on the disk. The directory contains a pointer to the first and last blocks of the file.



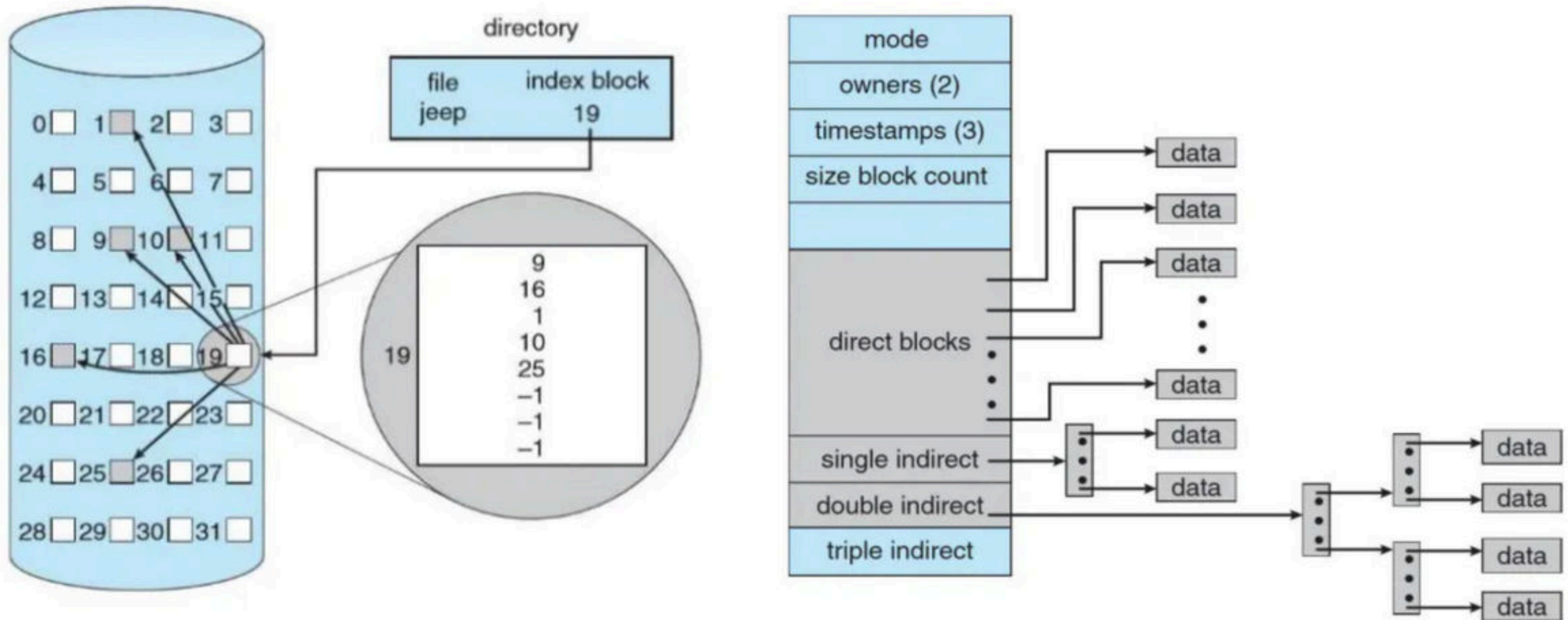
- **Advantage: -**
 - To create a new file, we simply create a new entry in the directory. With linked allocation, each directory entry has a pointer to the first disk block of the file. To read a file, we simply read blocks by following the pointers from block to block. The size of a file need not be declared when the file is created. A file can continue to grow as long as free blocks are available.
 - There is no external fragmentation with linked allocation, and any free block on the free-space list can be used to satisfy a request.

- **Disadvantage: -**
 - To find the i^{th} block of a file, we must start at the beginning of that file and follow the pointers until we get to the i^{th} block. Each access to a pointer requires a disk read.
 - Another disadvantage is the space required for the pointers, so each file requires slightly more space than it would otherwise.
 - Yet another problem is reliability. Recall that the files are linked together by pointers scattered all over the disk, and consider what would happen if a pointer were lost or damaged. One partial solution is to use doubly linked lists, and another is to store the file name and relative block number in each block. However, these schemes require even more overhead for each file.

Break

Indexed Allocation

- Indexed allocation solves problems of contiguous and linked allocation, by bringing all the pointers together into one location: the index block.



- Each file has its own index block, which is an array of disk-block addresses. The i^{th} entry in the index block points to the i^{th} block of the file. The directory entry contains the address of the index block. To find and read the i^{th} block, we use the pointer in the i^{th} index-block entry.
- When the file is created, all pointers in the index block are set to null. When the i^{th} block is first written, a block is obtained from the free-space manager, and its address is put in the i^{th} index-block entry.
- This point raises the question of how large the index block should be. Every file must have an index block, so we want the index block to be as small as possible. If the index block is too small, however, it will not be able to hold enough pointers for a large file.

- **Linked scheme:** To allow for large files, we can link together several index blocks. For example, an index block might contain a small header giving the name of the file and a set of the first 100 disk-block addresses. The next address (the last word in the index block) is null (for a small file) or is a pointer to another index block (for a large file).
- **Multilevel index.** A variant of linked representation uses a first-level index block to point to a set of second-level index blocks, which in turn point to the file blocks. To access a block, the operating system uses the first-level index to find a second-level index block and then uses that block to find the desired data block. This approach could be continued to a third or fourth level, depending on the desired maximum file size.

- **Combined scheme.** Another alternative, used in UNIX-based file systems, is to keep the first, say, 15 pointers of the index block in the file's inode.
- The first 12 of these pointers point to direct blocks; that is, they contain addresses of blocks that contain data of the file. Thus, the data for small files do not need a separate index block.
- The next three pointers point to indirect blocks. The first points to a single indirect block, which is an index block containing not data but the addresses of blocks that do contain data.
- The second points to a double indirect block, which contains the address of a block that contains the addresses of blocks that contain pointers to the actual data blocks. The last pointer contains the address of a triple indirect block.

- **Advantage**

- Indexed allocation supports direct access, without suffering from external fragmentation, because any free block on the disk can satisfy a request for more space.

- **Disadvantage**

- Indexed allocation does suffer from wasted space, however. The pointer overhead of the index block is generally greater than the pointer overhead of linked allocation.

Break

Q In a file allocation system, which of the following allocation scheme(s) can be used if no external fragmentation is allowed? **(GATE-2017) (1 Marks)**

I. Contiguous

II. Linked

III. Indexed

(a) I and III only

(b) II only

(c) III only

(d) II and III only

Q Consider a file currently consisting of 50 blocks. Assume that the file control block and the index block is already in memory. If a block is added at the end (and the block information to be added is stored in memory), then how many disk I/O operations are required for indexed (single-level) allocation strategy? **(NET-NOV-2016)**

A) 1

b) 101

c) 27

d) 0

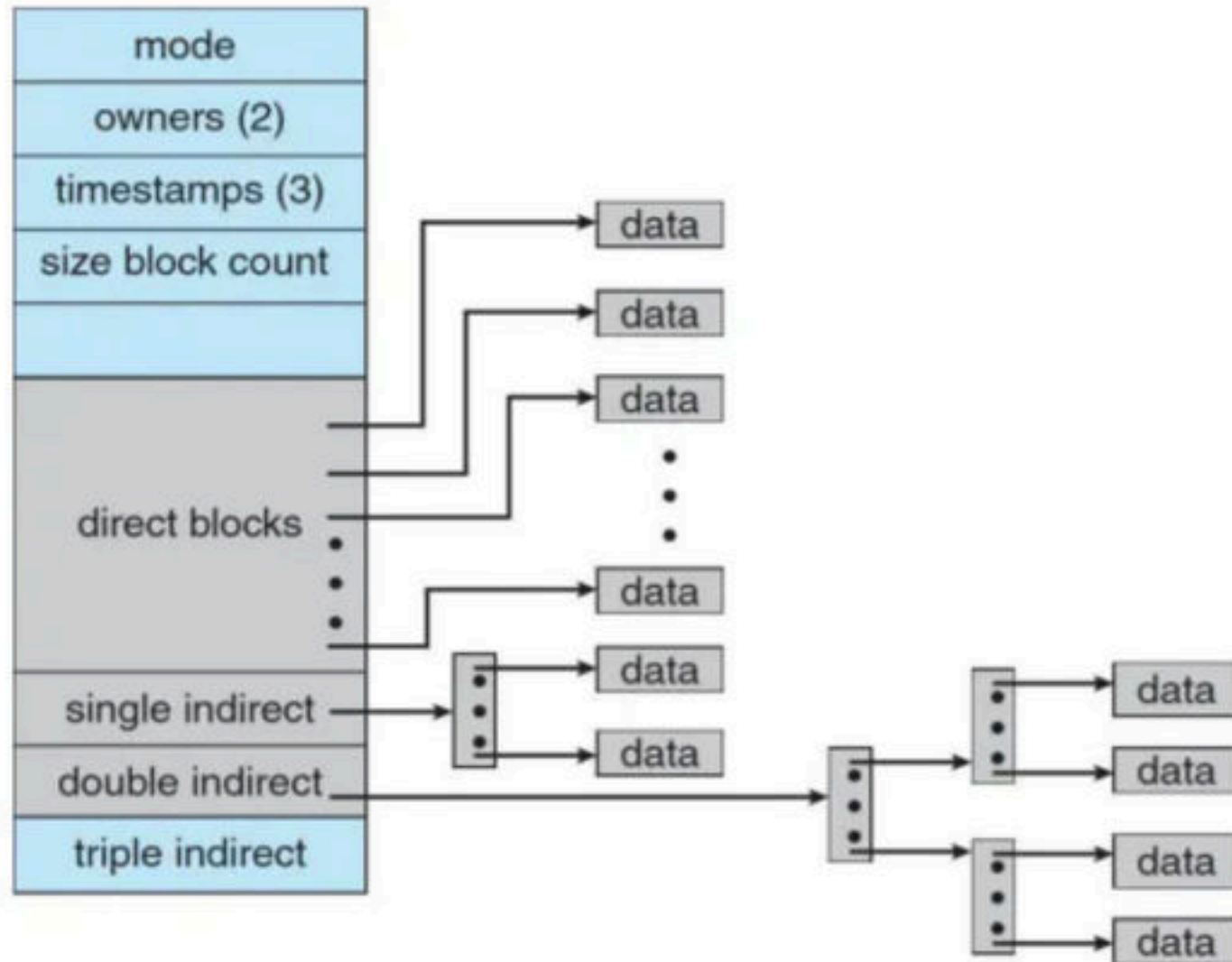
Q Match the following: (NET-JUNE-2014)

List-I	List-II
a) Contiguous allocation	i) This scheme supports very large file sizes.
b) Linked allocation	ii) This allocation technique supports only sequential files.
c) Indexed allocation	iii) Number of disks required to access file is minimal.
d) Multi-level indexed	iv) This technique suffers from maximum wastage of space in storing pointers.

	(a)	(b)	(c)	(d)
(a)	(iii)	(iv)	(ii)	(i)
(b)	(iii)	(ii)	(iv)	(i)
(c)	(i)	(ii)	(iv)	(iii)
(d)	(i)	(iv)	(ii)	(iii)

Use Referral Code **KGYT** for Unacademy Plus to Get minimum 10% Discount

Q The index node (inode) of a Unix-like file system has 12 direct, one single-indirect and one double-indirect pointers. The disk block size is 4 kB, and the disk block address is 32-bits long. The maximum possible file size is (rounded off to 1 decimal place) _____ GB. **(GATE-2014) (2 Marks)**



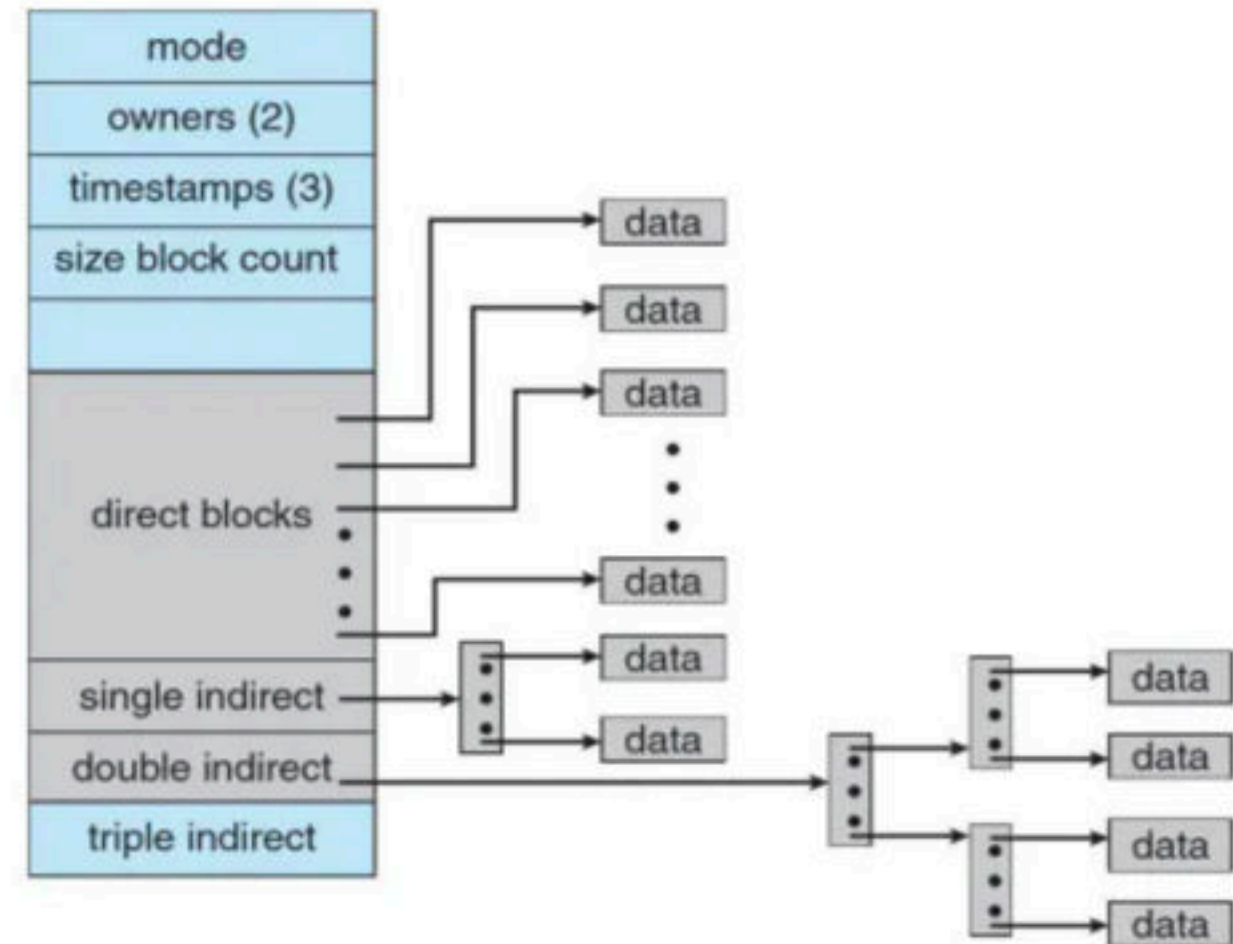
Q A file system with 300 Gbyte disk uses a file descriptor with 8 direct block addresses, 1 indirect block address and 1 doubly indirect block address. The size of each disk block is 128 Bytes and the size of each disk block address is 8 Bytes. The maximum possible file size in this file system is **(GATE-2012) (2 Marks)**

(A) 3 Kbytes

(B) 35 Kbytes

(C) 280 Bytes

(D) Dependent on the size of the disk



Q The data blocks of a very large file in the Unix file system are allocated using
(GATE-2008) (1 Marks)

(A) contiguous allocation

(B) linked allocation

(C) indexed allocation

(D) an extension of indexed allocation

Q A Unix-style i-node has 10 direct pointers and one single, one double and one triple indirect pointer. Disk block size is 1 Kbyte, disk block address is 32 bits, and 48-bit integers are used. What is the maximum possible file size? **(GATE-2004) (2 Marks)**

(A) 2^{24} bytes

(B) 2^{32} bytes

(C) 2^{34} bytes

(D) 2^{48} bytes

Use Referral Code KGYT for Unacademy Plus to Get minimum 10% Discount

Q In the index allocation scheme of blocks to a file, the maximum possible size of the file depends on **(GATE-2002) (1 Marks)**

- (a)** the size of the blocks, and the size of the address of the blocks.
- (b)** the number of blocks used for the index, and the size of the blocks.
- (c)** the size of the blocks, the number of blocks used for the index, and the size of the address of the blocks.
- (d)** None of the above

Break

Free-Space Management

- Since disk space is limited, we need to reuse the space from deleted files for new files, if possible.
- To keep track of free disk space, the system maintains a free-space list. The free-space list records all free disk blocks—those not allocated to some file or directory.
- To create a file, we search the free-space list for the required amount of space and allocate that space to the new file.
- This space is then removed from the free-space list. When a file is deleted, its disk space is added to the free-space list.

Q How many disk blocks are required to keep list of free disk blocks in a 16 GB hard disk with 1 kB block size using linked list of free disk blocks? Assume that the disk block number is stored in 32 bits. **(NET-DEC-2014)**

(A) 1024 blocks

(B) 16794 blocks

(C) 20000 blocks

(D) 1048576 blocks

Break

Bit Vector

- Frequently, the free-space list is implemented as a bit map or bit vector. Each block is represented by 1 bit. If the block is free, the bit is 1; if the block is allocated, the bit is 0.
- For example, consider a disk where blocks 2, 3, 4, 5, 8, 9, 10, 11, 12, 13, 17, 18, 25, 26, and 27 are free and the rest of the blocks are allocated. The free-space bit map would be 001111001111110001100000011100000 ...
- The main advantage of this approach is its relative simplicity and its efficiency in finding the first free block or n consecutive free blocks on the disk.

- Unfortunately, bit vectors are inefficient unless the entire vector is kept in main memory. Keeping it in main memory is possible for smaller disks but not necessarily for larger ones.
- A 1.3-GB disk with 512-byte blocks would need a bit map of over 332 KB to track its free blocks.
- A 1-TB disk with 4-KB blocks requires 256 MB to store its bit map. Given that disk size constantly increases, the problem with bit vectors will continue to escalate as well.

Q How much space will be required to store the bit map of a 1.3 GB disk with 512 bytes block size? (**NET-DEC-2013**)

(A) 332.8 KB

(B) 83.6 KB

(C) 266.2 KB

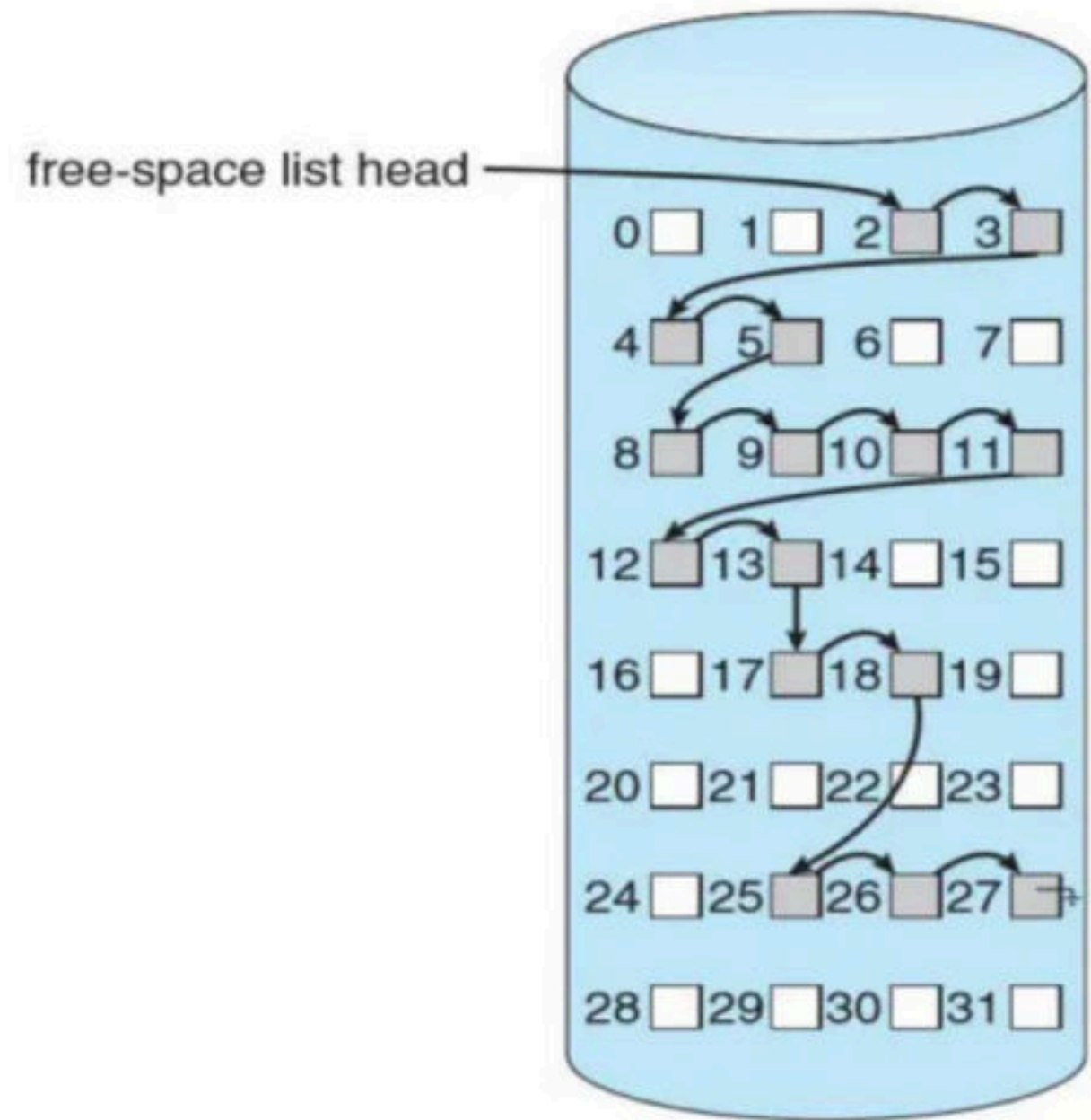
(D) 256.6 KB

Use Referral Code **KGYT** for Unacademy Plus to Get minimum 10% Discount

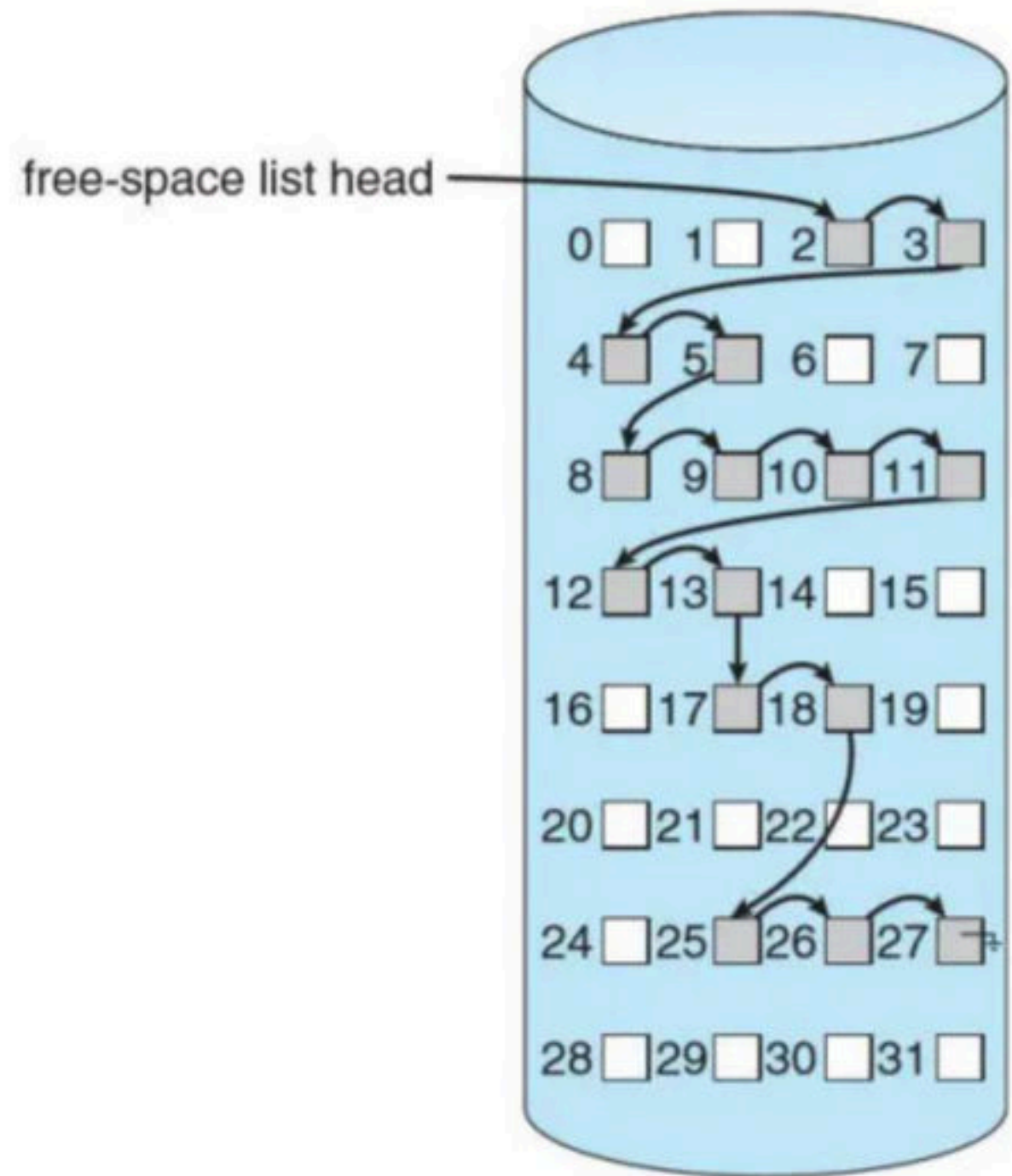
Break

Linked List

- Another approach to free-space management is to link together all the free disk blocks, keeping a pointer to the first free block in a special location on the disk and caching it in memory.
- This first block contains a pointer to the next free disk block, and so on. Recall our earlier example, in which blocks 2, 3, 4, 5, 8, 9, 10, 11, 12, 13, 17, 18, 25, 26, and 27 were free and the rest of the blocks were allocated.



- In this situation, we would keep a pointer to block 2 as the first free block. Block 2 would contain a pointer to block 3, which would point to block 4, which would point to block 5, which would point to block 8, and so on.
- This scheme is not efficient; to traverse the list, we must read each block, which requires substantial I/O time.
- Fortunately, however, traversing the free list is not a frequent action. Usually, the operating system simply needs a free block so that it can allocate that block to a file, so the first block in the free list is used.



Q A FAT (file allocation table) based file system is being used and the total overhead of each entry in the FAT is 4 bytes in size. Given a 100×10^6 bytes disk on which the file system is stored and data block size is 10^3 bytes, the maximum size of a file that can be stored on this disk in units of 10^6 bytes is _____. **(GATE-2014) (2 Marks)**

Break

Q An experimental file server is up 75% of the time and down for 25% of the time due to bugs. How many times does this file server have to be replicated to give an availability of at least 99%? **(NET-NOV-2016)**

a) 2

b) 4

c) 8

d) 16

Fork

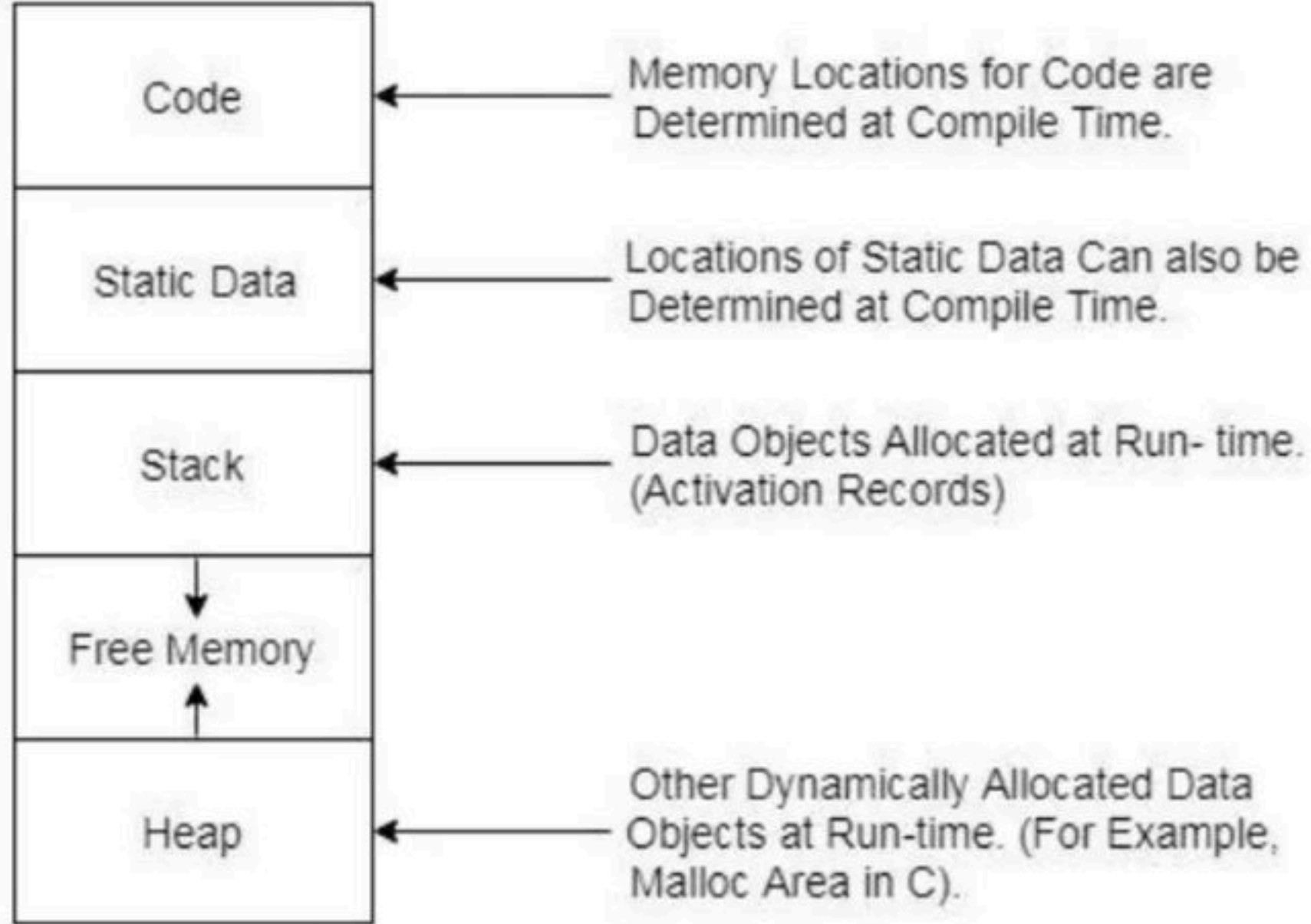
- Requirement of Fork command
 - In number of applications specially in those where work is of repetitive nature, like web server i.e. with every client we have to run similar type of code. Have to create a separate process every time for serving a new request.
 - So it must be a better solution that instead to creating a new process every time from scratch we must have a short command using which we can do this logic.

- **Idea of fork command**

- Here fork command is a system command using which the entire image of the process can be copied and we create a new process, this idea help us to complete the creation of the new process with speed.
- After creating a process, we must have a mechanism to identify weather in newly created process which one is child and which is parent.

- **Implementation of fork command**
 - In general, if fork return 0 then it is child and if fork return 1 then it is parent, and then using a programmer level code we can change the code of child process to behave as new process.

- **Advantages of using fork commands**
 - Now it is relatively easy to create and manage similar types of process of repetitive nature with the help of fork command.
- **Disadvantage**
 - To create a new process by fork command we have to do system call as, fork is system function
 - Which is slow and time taking
 - Increase the burden over Operating System
- Different image of the similar type of task have same code part which means we have the multiple copy of the same data waiting the main memory



Break

Q Fork is (ISRO 2008)

a) the creation of a new job

b) the dispatching of a task

c) increasing the priority of a task

d) the creation of a new process

Q The following C program is executed on a Unix/Linux system:

```
#include <unistd.h>
int main ()
{
    int i ;
    for (i=0; i<10; i++)
        if (i%2 == 0)
            fork ( ) ;
    return 0 ;
}
```

The total number of child processes created is _____ (**GATE-2019**) (1 Marks)

Q A process executes the code

`fork();`

`fork();`

`fork();`

the total number of child processes created is **(GATE - 2012) (1 Marks)**

a) 3

b) 4

c) 7

d) 8

Q A process executes the following code (**GATE - 2008**) (**2 Marks**)

```
for (i=0; i<n; i++)
```

```
fork();
```

a) n

b) $(2^n) - 1$

c) 2^n

d) $(2^{n+1}) - 1$

```
if (fork() == 0)
{
    a = a + 5;
    printf ("%d, %d /n", a, &a);
}
else
{
    a = a - 5;
    printf ("%d, %d /n", a, &a);
}
```

Let u, v be the values printed by the parent process, and x, y be the values printed by the child process. Which one of the following is TRUE? **(GATE-2005) (2 Marks)**

- a) $u = x + 10$ and $v = y$
- b) $u = x + 10$ and $v \neq y$
- c) $u + 10 = x$ and $v = y$
- d) $u + 10 = x$ and $v \neq y$

Q a process executes the following segment of code:

```
fork(i=1;i<=n;i++)
```

```
fork();
```

the number of new processes created is **(GATE-2004) (1 Marks)**

a) n

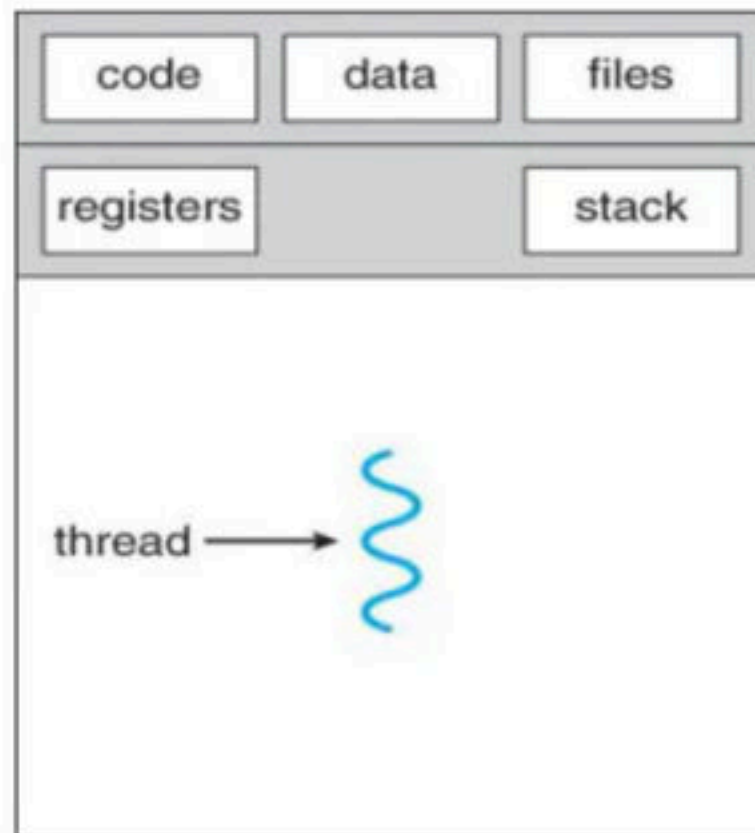
b) $(n(n+1))/2$

c) $2^n - 1$

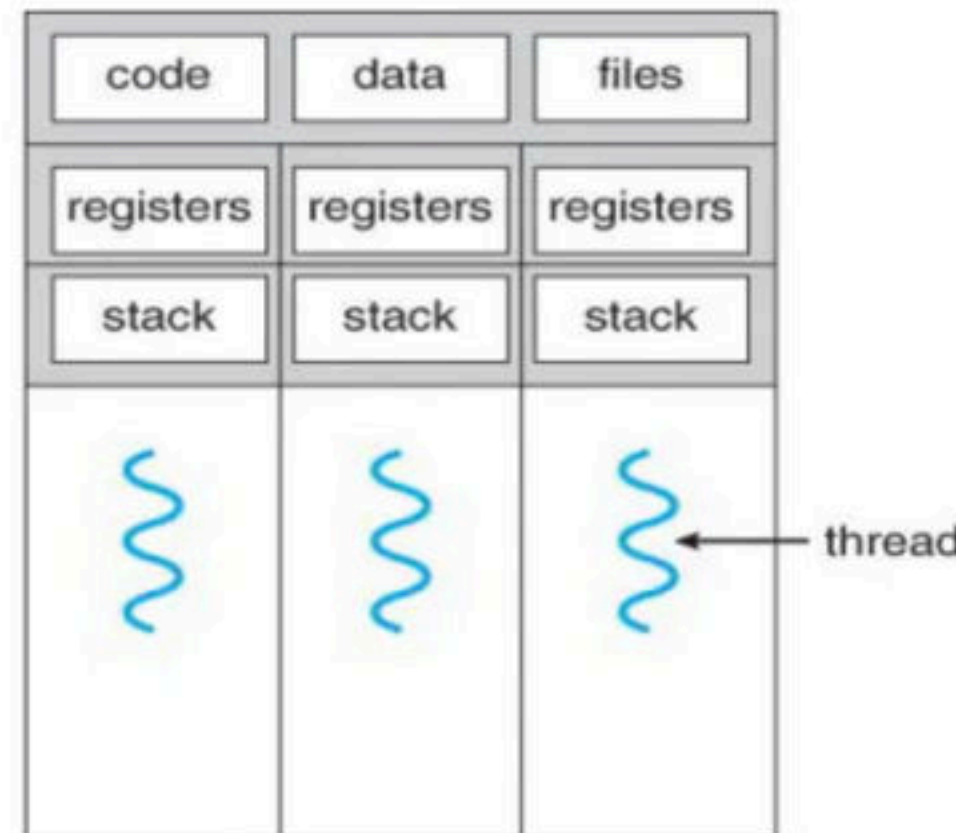
d) $3^n - 1$

Break

- A **thread** is a basic unit of CPU utilization, consisting of a program counter, a stack, and a set of registers, (and a thread ID.)
- Traditional (heavyweight) processes have a single thread of control - There is one program counter, and one sequence of instructions that can be carried out at any given time.
- multi-threaded applications have multiple threads within a single process, each having their own program counter, stack and set of registers, but sharing common code, data, and certain structures such as open files.



single-threaded process



multithreaded process

Motivation

- Threads are very useful in modern programming whenever a process has multiple tasks to perform independently of the others.
- This is particularly true when one of the tasks may block, and it is desired to allow the other tasks to proceed without blocking.
- For example in a word processor, a background thread may check spelling and grammar while a foreground thread processes user input (keystrokes), while yet a third thread loads images from the hard drive, and a fourth does periodic automatic backups of the file being edited.
- Another example is a web server - Multiple threads allow for multiple requests to be satisfied simultaneously.

There are four major categories of benefits to multi-threading:

- **Responsiveness** - One thread may provide rapid response while other threads are blocked or slowed down doing intensive calculations.
- **Resource sharing** - By default threads share common code, data, and other resources, which allows multiple tasks to be performed simultaneously in a single address space.
- **Economy** - Creating and managing threads (and context switches between them) is much faster than performing the same tasks for processes.
- **Scalability**, i.e. Utilization of multiprocessor architectures - A single threaded process can only run on one CPU, no matter how many may be available, whereas the execution of a multi-threaded application may be split amongst available processors. (Note that single threaded processes can still benefit from multi-processor architectures when there are multiple processes contending for the CPU, i.e. when the load average is above some certain threshold.)

Break

Q Which of the following is/are shared by all the threads in a process? (GATE - 2017) (1 Marks)

I. Program Counter

II. Stack

III. Address space

IV. Registers

(A) I and II only

(B) III only

(C) IV only

(D) III and IV only

Q Threads of a process share **(GATE - 2017) (1 Marks)**

- (A)** global variables but not heap
- (B)** heap but not global variables
- (C)** neither global variables nor heap
- (D)** both heap and global variables

Q A thread is a light weight process. In the above statement, weight refers to **(NET-DEC-2012)**

(A) time

(B) number of resources

(C) speed

(D) All the above

Q A thread is usually defined as a “light weight process” because an operating system (OS) maintains smaller data structures for a thread than for a process. In relation to this, which of the following is TRUE? **(GATE - 2011) (1 Marks)**

- (A)** On per-thread basis, the OS maintains only CPU register state
- (B)** The OS does not maintain a separate stack for each thread
- (C)** On per-thread basis, the OS does not maintain virtual memory state
- (D)** On per-thread basis, the OS maintains only scheduling and accounting information

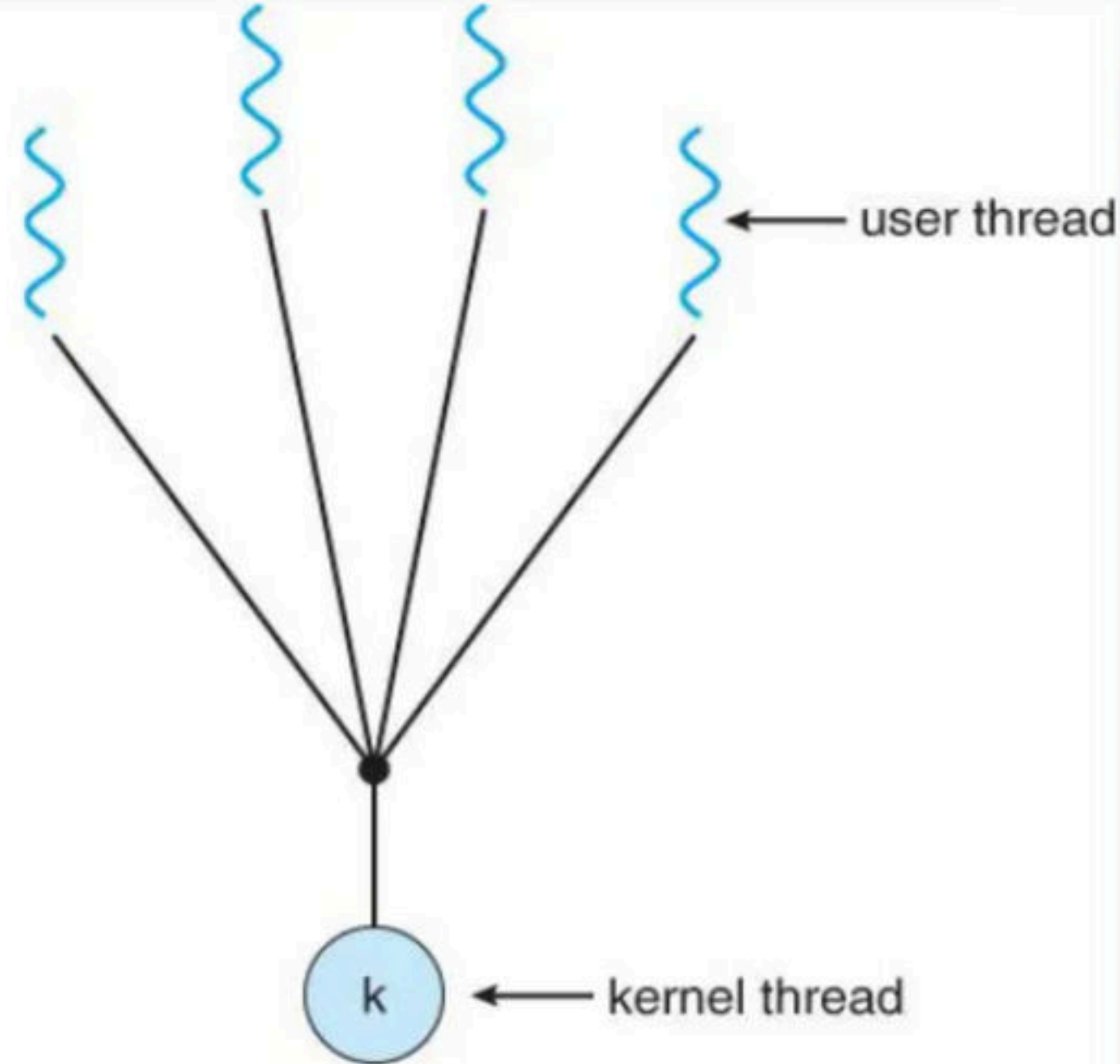
Break

Multithreading Models

- There are two types of threads to be managed in a modern system: User threads and kernel threads.
- User threads are supported above the kernel, without kernel support. These are the threads that application programmers would put into their programs.
- Kernel threads are supported within the kernel of the OS itself. All modern OS support kernel level threads, allowing the kernel to perform multiple simultaneous tasks and/or to service multiple kernel system calls simultaneously

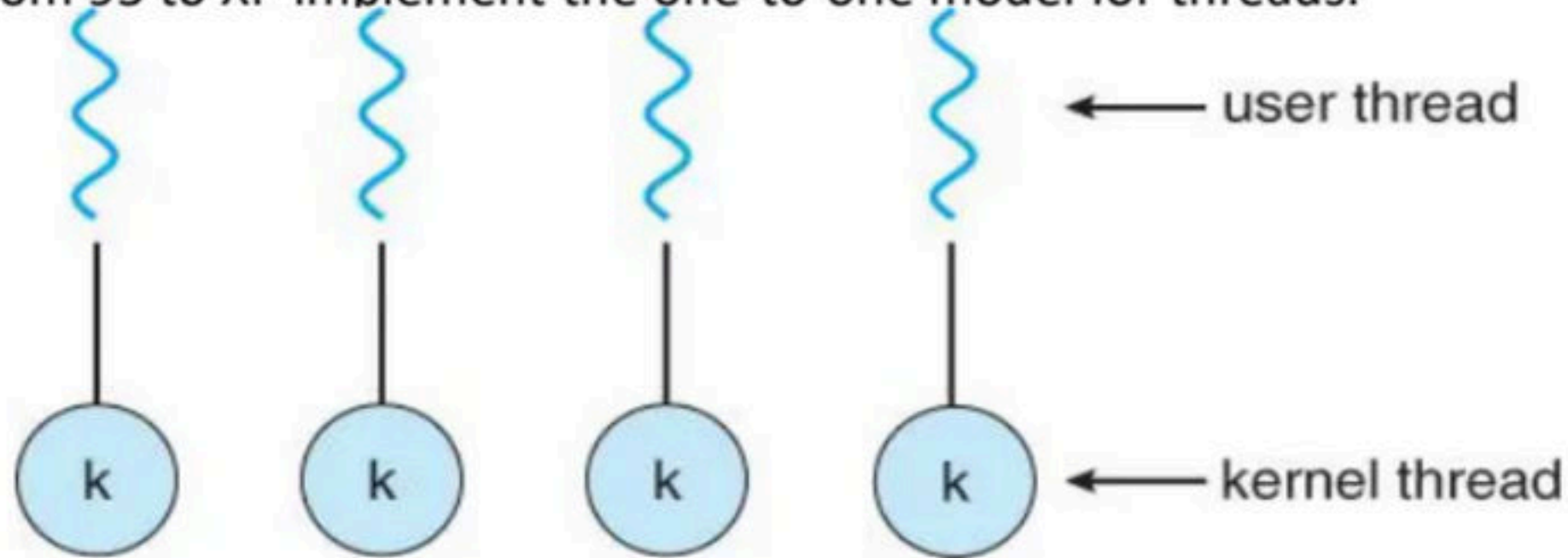
Many-To-One Model

- In the many-to-one model, many user-level threads are all mapped onto a single kernel thread.
- However, if a blocking system call is made, then the entire process blocks, even if the other user threads would otherwise be able to continue.
- Because a single kernel thread can operate only on a single CPU, the many-to-one model does not allow individual processes to be split across multiple CPUs.
- Green threads for Solaris implement the many-to-one model in the past, but few systems continue to do so today.



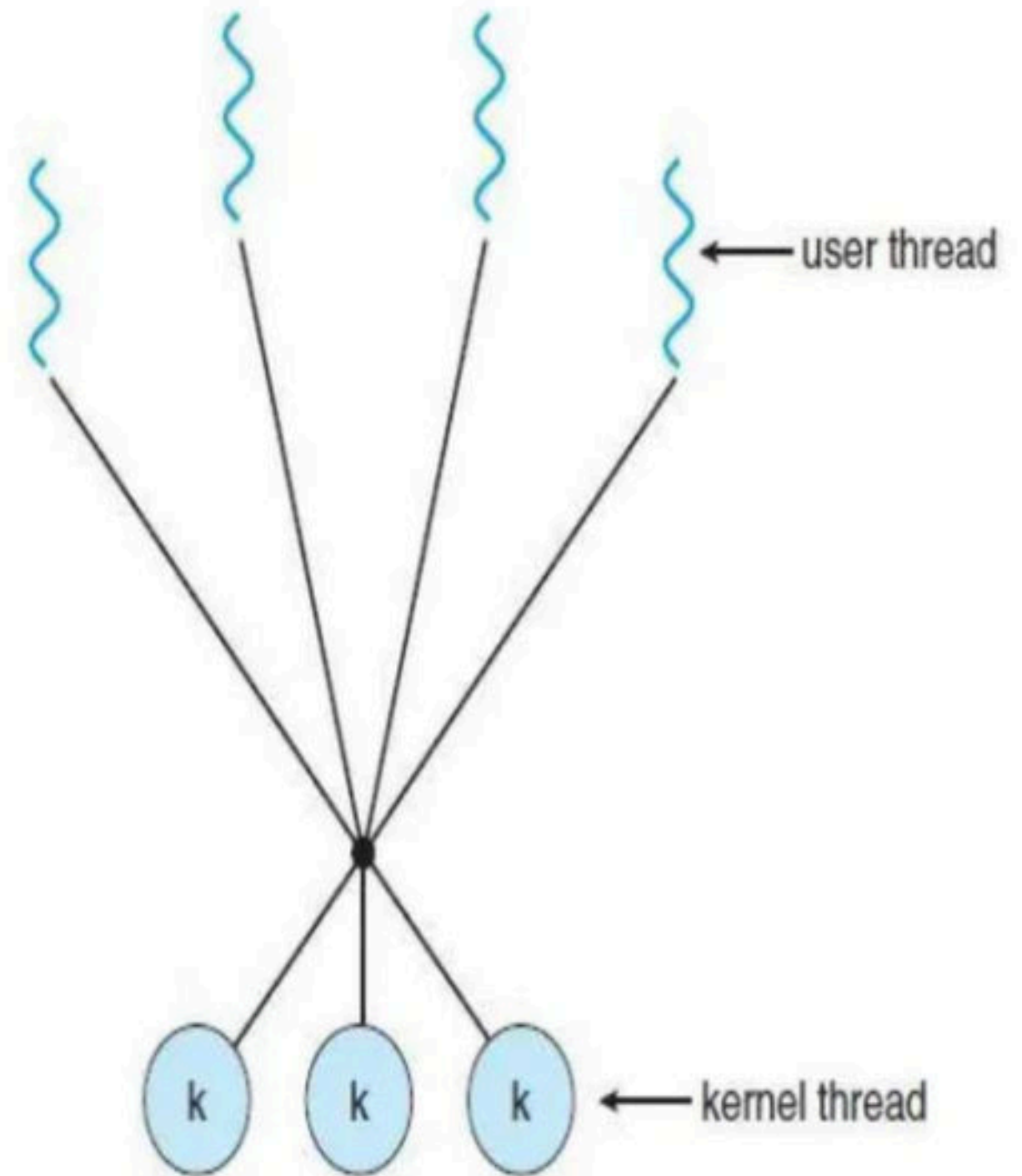
One-To-One Model

- The one-to-one model creates a separate kernel thread to handle each user thread.
- One-to-one model overcomes the problems listed above involving blocking system calls and the splitting of processes across multiple CPUs.
- However, the overhead of managing the one-to-one model is more significant, involving more overhead and slowing down the system.
- Most implementations of this model place a limit on how many threads can be created.
- Linux and Windows from 95 to XP implement the one-to-one model for threads.



Many-To-Many Model

- The many-to-many model multiplexes any number of user threads onto an equal or smaller number of kernel threads, combining the best features of the one-to-one and many-to-one models.
- Users have no restrictions on the number of threads created.
- Blocking kernel system calls do not block the entire process.
- Processes can be split across multiple processors.
- Individual processes may be allocated variable numbers of kernel threads, depending on the number of CPUs present and other factors.



Break

Q User level threads are threads that are visible to the programmer and are unknown to the kernel. The operating system kernel supports and manages kernel level threads. Three different types of models relate user and kernel level threads. Which of the following statements is/are true? **(NET-NOV-2017)**

(a) (i) The Many - to - one model maps many user threads to one kernel thread
(ii) The one - to - one model maps one user thread to one kernel thread
(iii) The many - to - many model maps many user threads to smaller or equal kernel threads

(b) (i) Many - to - one model maps many kernel threads to one user thread
(ii) One - to - one model maps one kernel thread to one user thread
(iii) Many - to - many model maps many kernel threads to smaller or equal user threads

A) (a) is true; (b) is false

B) (a) is false; (b) is true

C) Both (a) and (b) are true

D) Both (a) and (b) are false

Use Referral Code KGYT for Unacademy Plus to Get minimum 10% Discount

Q One of the disadvantages of user level threads compared to Kernel level threads is (**NET-JAN-2017**)

- a)** If a user level thread of a process executes a system call, all threads in that process are blocked.
- b)** Scheduling is application dependent.
- c)** Thread switching doesn't require kernel mode privileges.
- d)** The library procedures invoked for thread management in user level threads are local procedures.

Q Which one of the following is FALSE? (GATE - 2014) (1 Marks)

(A) User level threads are not scheduled by the kernel.

(B) When a user level thread is blocked, all other threads of its process are blocked.

(C) Context switching between user level threads is faster than context switching between kernel level threads.

(D) Kernel level threads cannot share the code segment

Q Let the time taken to switch between user mode and kernel mode of execution be T_1 while time taken to switch between two user processes be T_2 . Which of the following is correct? (**NET-JUNE-2013**)

a) $T_1 < T_2$

b) $T_1 > T_2$

c) $T_1 = T_2$

d) Nothing can be said about the relation between T_1 and T_2 .

Q Let the time taken to switch between user and kernel modes of execution be t_1 while the time taken to switch between two processes be t_2 . Which of the following is TRUE? (**GATE-2011**) (1 Marks)

(A) $t_1 > t_2$

(B) $t_1 = t_2$

(C) $t_1 < t_2$

(D) nothing can be said about the relation between t_1 and t_2

Q Consider the following statements about user level threads and kernel level threads. Which one of the following statements is FALSE? **(GATE - 2007) (1 Marks)**

A) Context switch time is longer for kernel level threads than for user level threads.

B) User level threads do not need any hardware support.

C) Related kernel level threads can be scheduled on different processors in a multi-processor system

D) Blocking one kernel level thread blocks all related threads

Q Consider the following statements with respect to user-level threads and kernel supported threads

- i. context switch is faster with kernel-supported threads
- ii. for user-level threads, a system call can block the entire process
- iii. Kernel supported threads can be scheduled independently
- iv. User level threads are transparent to the kernel

Which of the above statements are true? **(GATE-2004) (1 Marks)**

(A) (ii), (iii) and (iv) only

(B) (ii) and (iii) only

(C) (i) and (iii) only

(D) (i) and (ii) only